GitForce

Guide de démarrage et d'utilisation

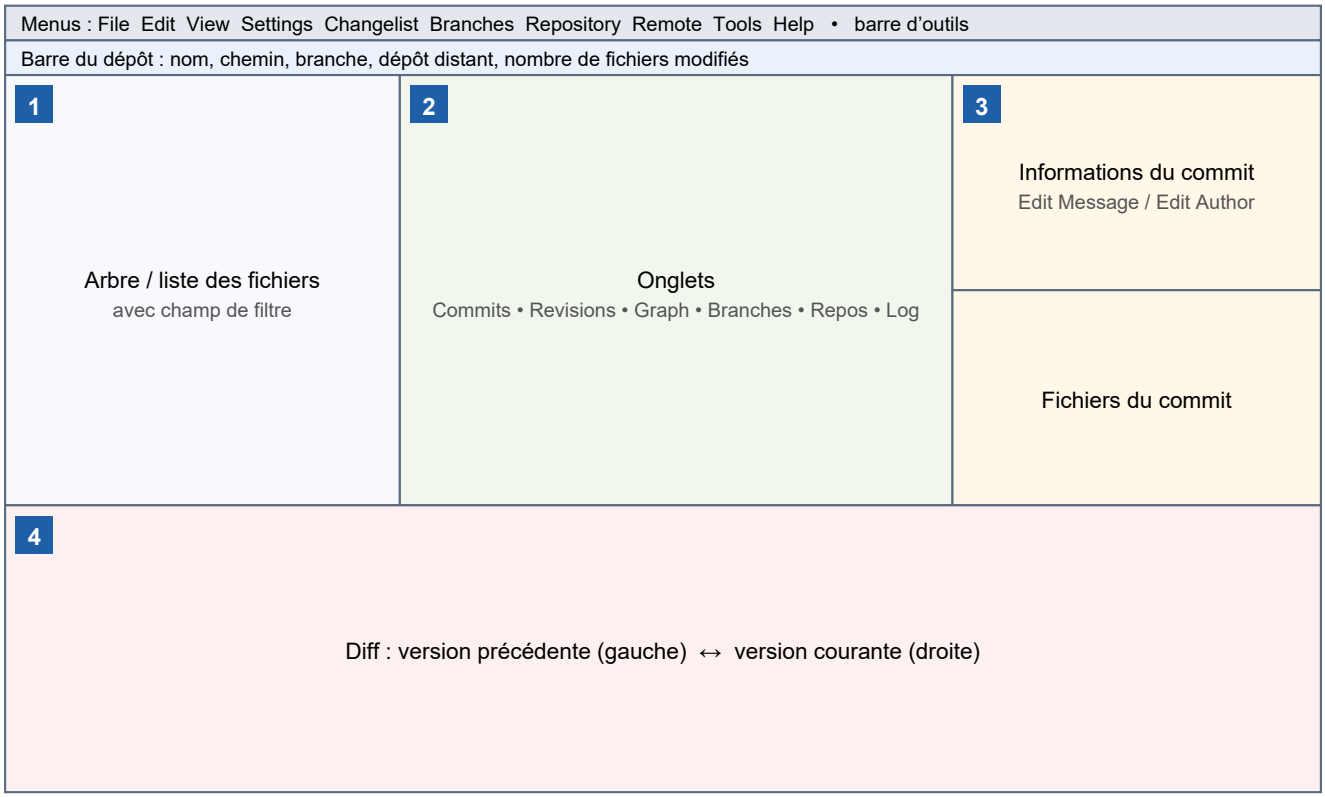
GitForce est une interface graphique pour l'outil en ligne de commande **git**. Ce guide décrit la fenêtre principale et les opérations de cette version : consulter l'état des fichiers, les indexer, les mettre de côté (stash), valider des modifications, parcourir et corriger l'historique, comparer des versions, gérer les branches, les tags et les dépôts distants.

Presque toutes les commandes sont disponibles de plusieurs façons : par le menu principal, par un clic droit (menu contextuel), par la barre d'outils et souvent par glisser-déposer. Les noms des commandes sont en anglais dans le programme ; ce guide les cite tels qu'ils apparaissent à l'écran.

Sommaire

1. La fenêtre principale
2. Les fichiers (1)
3. Les onglets (2)
4. Le détail d'un commit (3)
5. Le panneau de diff (4)
6. Valider des modifications
7. Mettre de côté : stash et unstash
8. Branches, fusion et conflits
9. Corriger l'historique et gérer les tags
10. Historique d'un fichier et blame
11. Le visualiseur de fichiers
12. Dépôts distants
13. Commandes Git et ligne de commande
14. Apparence et réglages
15. Raccourcis clavier
16. Aide et signalement des bugs

1. La fenêtre principale



- **Barre du dépôt** (sous la barre d'outils) : rappelle le dépôt en cours, son dossier, la branche courante, le dépôt distant utilisé et le nombre de fichiers modifiés. Le titre de la fenêtre reprend ces informations.
- **(1) Fichiers** : les fichiers du dépôt, en arbre ou en liste, avec une icône qui indique leur état git.
- **(2) Onglets** : modifications en attente, historique, graphe, branches, dépôts et log des commandes git.
- **(3) Détail du commit** : en haut les informations du commit choisi dans Revisions ou Graph, en bas la liste de ses fichiers.
- **(4) Diff** : le fichier sélectionné, version précédente à gauche et version courante à droite.

Les séparateurs entre les panneaux se déplacent à la souris ; leurs positions sont mémorisées. Avec **View > Auto Refresh** (activé par défaut), les vues se mettent à jour dès qu'un fichier du dépôt change, y compris quand la modification vient d'un autre programme ; `F5` force le rafraîchissement.

2. Les fichiers (1)

Les boutons en haut du panneau basculent entre arbre et liste, rafraîchissent la vue et choisissent le mode d'affichage (également dans le menu **View**) :

Mode	Ce qui est affiché
Git Status of All Files `Alt+1`	Les fichiers modifiés, ajoutés, supprimés et non suivis.
Git Status `Alt+2`	La même chose, sans les fichiers non suivis.
Git View of Repo `Alt+3`	Le contenu du dernier commit, indépendamment des fichiers locaux.
Local File View `Alt+4`	Tous les fichiers présents sur le disque.
Local Files Not in Repo `Alt+5`	Les fichiers qui ne sont pas encore dans le dépôt.
Local Files Not in .gitignore `Alt+6`	Les fichiers locaux qui ne sont pas exclus par .gitignore.
Ignored Files	Uniquement les fichiers et dossiers ignorés par .gitignore, pour les réintégrer au besoin.

Mode	Ce qui est affiché
Changed Files (flat list)	Tous les fichiers suivis modifiés, indexés ou non, en une seule liste avec leur chemin complet. Les fichiers non suivis n'y figurent pas.

Le même menu trie les fichiers par extension et, avec **Show Ignored Files**, affiche ou masque les fichiers ignorés dans les autres vues.

Filtrer les fichiers

Le champ de filtre, en haut du panneau, ne garde que les fichiers qui correspondent. Plusieurs critères peuvent être combinés, séparés par des espaces ; `Échap` ou la croix vide le filtre.

Saisie	Effet
`*.cs` ou `.cs`	Fichiers de cette extension (les jokers `*` et `?` sont admis).
`Panel`	Fichiers dont le nom ou le chemin contient ce texte.
`!test`	Exclut ce qui contient ce texte.
`*.cs !Designer`	Les fichiers C#, sauf ceux des formulaires générés.

Icônes d'état

Icône	Signification
Point d'interrogation	Fichier non suivi (untracked) : git ne le connaît pas encore.
Crayon / triangle	Fichier modifié.
Plus	Fichier ajouté à l'index.
Croix	Fichier supprimé.
Signe d'interdiction rouge, élément grisé	Fichier ou dossier ignoré par .gitignore.
Dossier marqué	Le dossier contient des modifications.

Avec **View > Toolbar Icons > Modern style**, ces icônes sont dessinées au trait, adaptées aux thèmes clair et sombre ; sinon, ce sont les icônes d'origine de GitForce.

Opérations sur les fichiers et dossiers

Sélectionnez un ou plusieurs éléments, puis faites un clic droit (ou utilisez le menu **File**) :

Commande	Effet
Stage / Unstage	Met dans l'index, ou en retire, toutes les modifications des éléments sélectionnés (nouveaux fichiers et suppressions compris). Les fichiers sur le disque ne changent pas.
Ignore (add to .gitignore)	Ajoute les fichiers, et les dossiers avec tout leur contenu, à .gitignore. Si des fichiers sont déjà suivis, GitForce propose de ne plus les suivre en les gardant sur le disque.
Ignore *.ext	Ignore tous les fichiers ayant l'extension du fichier sélectionné (par exemple `*.bak`).
Ignore folder	Ignore le dossier du fichier sélectionné, avec tout son contenu.
Unignore (make untracked)	Retire la règle .gitignore correspondante : l'élément redevient non suivi.
Untrack (keep on disk)	Arrête le suivi d'un fichier suivi sans le supprimer du disque ; il réapparaît comme non suivi (?). Le retrait devient définitif au prochain commit.
Stash Selected...	Met de côté les modifications des éléments sélectionnés (section 7).
Blame	Pour chaque ligne du fichier, le commit, l'auteur et la date de sa dernière modification (section 10).

Commande	Effet
Update Changelist, Revert, Open for Delete, Move/Rename, Remove from File System	Les opérations habituelles sur les fichiers suivis.
Diff vs	Compare le fichier à l'index (`Ctrl+D`) ou au dernier commit (`Ctrl+Shift+D`).
Edit Using	Ouvre le fichier avec l'éditeur associé, un éditeur de votre choix ou le visualiseur intégré (section 11).
Revision History...	L'historique du fichier avec sa comparaison côte à côte (section 10), `Ctrl+H`.
Explore..., Command Prompt...	Ouvre le dossier dans l'Explorateur ou une invite de commandes.

Un clic sur un fichier l'affiche dans le panneau de diff (4). L'action du double-clic se règle dans **Settings > Options** : ne rien faire, ouvrir avec l'application associée, avec un programme choisi ou avec le visualiseur intégré.

3. Les onglets (2)

Commits — modifications en attente `F6`

Une *changelist* regroupe des fichiers prêts à être validés. Glissez des fichiers depuis le panneau de gauche vers une changelist pour les y ajouter. Vous pouvez créer plusieurs changelists, déplacer des fichiers de l'une à l'autre, puis soumettre chacune séparément (**Submit...**) : GitForce ne valide que les fichiers de la liste soumise. Un clic sur un fichier l'affiche dans le diff ; le clic droit propose aussi **Unstage**, **Blame**, **Revision History** et **Diff vs Repo HEAD**.

Revisions — historique `F7`

L'historique de la branche courante. Un **clic** sur un commit affiche ses informations et ses fichiers à droite (3) ; un **double-clic** ouvre **Describe Changelist**. Le clic droit permet de copier le SHA, de colorer des commits, de faire un cherry-pick ou un reset, de poser un tag et de modifier le message ou l'auteur d'un commit (section 9).

Graph — graphe des commits `F9`

Les commits sous forme de graphe, comme dans SmartGit ou Tower : chaque branche a sa couleur, les fusions sont reliées, les branches, tags et HEAD sont affichés en étiquettes. Un bouton choisit entre toutes les branches et la seule branche courante. Les **stashes** apparaissent comme des branches spéciales `stash@{n}`. La largeur des colonnes se règle à la souris et est mémorisée.

Un clic sur un commit ou un stash affiche ses fichiers à droite. Le clic droit permet de basculer sur une branche, de la fusionner, de la renommer, de créer une branche ou un tag à cet endroit, de renommer ou supprimer un tag, de modifier le message ou l'auteur, de faire un cherry-pick ou un reset (section 9), et, sur un stash, de l'appliquer (**Apply**), de le récupérer (**Pop**) ou de le supprimer (**Drop**).

Branches `F8`

Les branches locales et distantes. En face de chaque branche : l'identifiant court de son dernier commit, ses tags et le sujet du commit. Un double-clic sur une branche locale bascule dessus ; le clic droit permet de créer, renommer, supprimer, basculer et fusionner (section 8).

Repos — dépôts `F10`

Les dépôts de l'espace de travail (*workspace*). Un marqueur indique le dépôt courant, un autre celui qui s'ouvre par défaut. Un double-clic rend un dépôt courant. Le clic droit permet de créer, cloner, rechercher (**Scan**), modifier ou retirer des dépôts et de les regrouper en projets. Les espaces de travail se créent, se chargent et s'enregistrent depuis le menu **File**.

Log — commandes git `F12`

Chaque commande git lancée par GitForce s'affiche ici avec son résultat ; les erreurs apparaissent en rouge. Cet onglet est surtout utile pour comprendre un problème. La ligne de commande (`F3`) s'y trouve aussi (section 13).

4. Le détail d'un commit (3)

Quand un commit est sélectionné dans Revisions ou Graph, la partie droite de la fenêtre affiche :

- **en haut**, ses informations : SHA, auteur, date, branches et tags, message complet ; les boutons **Edit Message...** et **Edit Author...** permettent de les corriger (section 9) ;
- **en bas**, la liste de ses fichiers avec leur type de modification. Un clic sur un fichier l'affiche dans le diff (4), comparé à sa version dans le commit précédent ; si le fichier n'existait pas avant, la partie gauche reste vide.

Le clic droit sur un fichier de la liste propose :

Commande	Effet
File History	L'historique de ce fichier (section 10).
Blame	Qui a modifié chaque ligne, dans l'état de ce commit.
Open in Built-in Viewer / Open in Editor...	Ouvre la version du fichier de ce commit.
Open Containing Folder...	Ouvre le dossier du fichier dans l'Explorateur.
Copy File Path	Copie le chemin du fichier.
Revert File	Annule dans la copie de travail les modifications que ce commit a apportées au fichier.
Cherry Pick File	Applique à la copie de travail les modifications de ce commit sur ce seul fichier.
Save Patch...	Enregistre les modifications du fichier dans ce commit sous forme de patch.

5. Le panneau de diff (4)

Le fichier sélectionné (dans les fichiers, une changelist ou un commit) s'affiche aussitôt en entier :

- à **gauche**, la version précédente ; à **droite**, la version courante ;
- les lignes ajoutées sont sur fond vert, les lignes supprimées sur fond rouge, avec les numéros de ligne des deux côtés ;
- pour un fichier sans version précédente (nouveau ou non suivi), la partie gauche reste vide ;
- **Side by side** bascule entre la vue côte à côte et la vue unifiée. En côte à côte, les deux versions tiennent dans la largeur du panneau (la fin des très longues lignes est coupée) ; la vue unifiée permet de les faire défiler ;
- **Previous change / Next change** (`Shift+F7` / `F7`) passe d'une modification à l'autre ; **Refresh** recharge les fichiers ; le compteur `+n -n` résume les lignes ajoutées et retirées ;
- le code C#, C, C++, Python, Java, Rust, Go et Pascal est colorisé (mots-clés, types, chaînes, commentaires, nombres, directives et annotations).

Les commandes **Diff vs** ouvrent le même comparateur dans une fenêtre séparée. Si un outil externe (WinMerge, Meld, KDiff3…) est configuré dans **Settings > Options**, ces commandes l'utilisent à la place.

6. Valider des modifications

- Indexez les fichiers (**Stage**, ou glisser-déposer vers une changelist), vérifiez-les dans le diff, puis **Submit...** sur la changelist.

- La fenêtre de commit liste les fichiers (à cocher ou décocher) et indique la longueur de la première ligne et du corps du message.
- **Amend** : cochez la case pour remplacer le dernier commit. Son message est repris automatiquement dans la zone de texte, prêt à être gardé tel quel ou modifié ; si vous décochez sans l'avoir modifié, votre texte précédent revient.

Amend réécrit le dernier commit : évitez-le si ce commit a déjà été poussé et que d'autres personnes l'utilisent.

7. Mettre de côté : stash et unstash

Le stash met de côté des modifications en cours pour les réappliquer plus tard, par exemple pour changer de branche.

Stash

Par **Edit > Stash** (tous les fichiers) ou **Stash Selected...** sur une sélection. La fenêtre permet de choisir les fichiers, de saisir un message et de choisir le mode :

Mode	Effet
Discard	Les modifications sont enregistrées puis retirées des fichiers de travail (comportement habituel de git stash).
Keep index	Comme Discard, mais les modifications indexées restent dans l'index.
Keep all	Les modifications sont enregistrées et restent dans les fichiers : une simple sauvegarde.

La case **Include untracked (new) files** inclut les fichiers non suivis.

Unstash

Edit > Unstash ouvre la liste des stashes avec, pour chacun, ses fichiers. On peut :

- **Apply** : réappliquer le stash en le gardant ; **Pop** : le réappliquer et le supprimer ;
- **Restore staged state** : remettre aussi dans l'index ce qui y était ;
- **Drop** : supprimer un stash ; **Edit message...** : renommer un stash ;
- **New branch...** : créer une branche à partir du stash et l'y appliquer.

Les stashes apparaissent aussi dans l'onglet Graph, où un clic montre leurs fichiers.

8. Branches, fusion et conflits

- **Créer** : **Branches > New...**, ou **New Branch Here...** sur un commit du graphe.
- **Basculer** : double-clic sur une branche, ou **Switch to**.
- **Renommer** : **Rename...** dans l'onglet Branches ou dans le graphe.
- **Fusionner** : **Merge with...**, ou **Merge into** dans le graphe. La fusion est faite par GitForce lui-même, sans outil externe.

Résoudre les conflits

Quand une fusion produit des conflits, les fichiers concernés apparaissent dans l'onglet Commits. **Resolve** ouvre le résolveur intégré : pour chaque conflit, choisissez la version de la branche courante (**Ours**, `Alt+1`), celle de la branche fusionnée (**Theirs**, `Alt+2`), les deux dans un ordre ou dans l'autre, ou modifiez le résultat à la main. `F7` / `Shift+F7` passent d'un conflit à l'autre ; **Save and mark resolved** enregistre le fichier. **Accept ours** / **Accept theirs** règlent un fichier entier d'un coup, et **Abort Merge** abandonne la fusion. Terminez par **Submit Merge...**.

9. Corriger l'historique et gérer les tags

Depuis Revisions, Graph ou le détail du commit :

- **Edit Commit Message...** : modifie le message d'un commit, y compris le dernier ;
- **Edit Commit Author...** : modifie le nom et l'e-mail de l'auteur, et au besoin du committer (**Also set as committer**) ;
- **New Tag Here...**, **Rename Tag...**, **Delete Tag** : gère les tags, légers ou annotés.

Modifier un commit ancien réécrit ce commit et tous ceux qui le suivent : leurs identifiants changent. Si ces commits ont déjà été poussés, il faudra forcer le push, ce qui gêne les autres personnes qui les utilisent.

Cherry-pick et reset

Depuis Revisions ou Graph, sur un commit :

- **Cherry-Pick...** : applique les modifications de ce commit à la branche courante, en créant un nouveau commit ou seulement dans les fichiers, pour les relire avant de valider. En cas de conflit, le résolveur s'ouvre (section 8).
- **Reset...** : ramène la branche courante à ce commit, en mode soft, mixed ou hard.
- **Reset Advanced...** : la même chose avec les cinq modes de git, chacun expliqué dans la fenêtre :

Mode	Index (fichiers indexés)	Fichiers de travail
mixed (par défaut)	Remis à l'état du commit	Gardent leurs modifications
soft	Inchangé : il contient toutes les différences, prêtes à être validées à nouveau	Inchangés
hard	Remis à l'état du commit	**Remis à l'état du commit : les modifications sont perdues** (confirmation demandée)
keep	Remis à l'état du commit	Gardent leurs modifications ; le reset est annulé si un fichier modifié devait être remplacé
merge	Remis à l'état du commit	Comme hard, mais le reset est annulé si un fichier modifié devait être remplacé

En mode hard, **Thoroughly fix line endings** extrait à nouveau tous les fichiers pour leur donner les fins de ligne demandées par .gitattributes.

10. Historique d'un fichier et blame

Revision History `Ctrl+H`

La fenêtre d'historique liste les commits qui ont modifié le fichier, avec la description du commit choisi. À droite, le fichier s'affiche **côte à côte** :

- avec **un** commit sélectionné, sa version est comparée à la version précédente du fichier ; il suffit de parcourir la liste avec les flèches pour voir l'évolution du fichier commit après commit ;
- avec **deux** commits sélectionnés (`Ctrl`+clic), le plus ancien est à gauche et le plus récent à droite ;
- **Diff vs. Client file** compare une révision au fichier actuel sur le disque ; **Diff Revisions** compare deux révisions dans une fenêtre séparée ; **View** ouvre une révision et **Sync** remet le fichier dans cet état.

Blame

La fenêtre Blame montre, pour chaque ligne du fichier, le commit, l'auteur et la date de sa dernière modification ; les lignes d'un même commit sont regroupées. Un double-clic sur une ligne décrit le commit correspondant.

11. Le visualiseur de fichiers

Par **Edit Using > Built-in Viewer**, **Open in Built-in Viewer** ou le double-clic s'il est réglé ainsi, il affiche :

- les fichiers texte et le code source avec numéros de ligne et colorisation (C#, C, C++, Python, Java, Rust, Go, Pascal), avec recherche (`Ctrl+F`, puis `F3`) ;
- les images PNG, JPG, GIF, BMP, ICO et TIF avec zoom ;
- pour les autres fichiers binaires, un bouton **Open externally** qui les ouvre avec l'application associée.

12. Dépôts distants

Le menu **Remote** regroupe les échanges avec les dépôts distants :

- **Fetch from Remote**, **Pull from Remote**, **Push to Remote** : récupère ou envoie les commits. Dans la barre d'outils, maintenir `Ctrl` en cliquant sur Pull ou Push applique l'opération à tous les dépôts ;
- **Switch Remote Repo...** : choisit le dépôt distant utilisé quand il y en a plusieurs ;
- **Edit Remote Repos...** : ajoute, modifie ou supprime des dépôts distants.

Les clés SSH (**Manage SSH Keys...**) et les mots de passe HTTPS (**Manage HTTPS Pass...**) se gèrent dans le menu **Settings**.

13. Commandes Git et ligne de commande

Fenêtre Commandes Git `Ctrl+Shift+G`

Tools > Git Commands... ouvre un aide-mémoire de plus de 120 commandes git classées par thème (création et clonage, état, index, commits, historique, branches et fusion, dépôts distants, tags, stash, annulation, réécriture avancée de l'historique, sous-modules et worktrees, recherche d'un bug avec bisect, configuration, maintenance), chacune avec une courte explication, présentées en accordéon. On peut aussi les exécuter directement :

- le champ du haut filtre les commandes (par exemple `stash` ou `annuler`) ;
- un clic met la commande dans la ligne `git`, avec la première `<valeur>` sélectionnée pour être remplacée ; un double-clic ou `Entrée` l'exécute dans le dépôt courant et la sortie s'affiche à droite ;
- **Arrêter** interrompt une commande en cours, **Effacer** vide la sortie, les flèches ↑ / ↓ rappellent les commandes déjà lancées ;
- les commandes marquées ⚠ peuvent perdre des modifications ou réécrire l'historique : GitForce demande confirmation ;
- après une commande qui modifie le dépôt, la fenêtre principale se met à jour.

Ligne de commande `F3`

View > Command Line affiche une ligne de commande dans l'onglet Log, pour taper une commande git sans quitter la fenêtre principale.

14. Apparence et réglages

- **View > Theme** : thème clair, sombre ou celui du système ; GitForce propose de redémarrer pour l'appliquer.
- **View > Toolbar Icons** : icônes petites (16), moyennes (24) ou grandes (32), et **Modern style** pour les icônes au trait.
- **View > Auto Refresh** : rafraîchissement automatique quand des fichiers changent.
- **Settings > Options** : chemin de git, outils de diff et de merge, éditeurs, action du double-clic, nombre de commits affichés, polices, etc.
- **Tools** : ajoutez vos propres outils (**Customize**), importés ou exportés avec **Import** / **Export** ; ils apparaissent aussi dans les menus contextuels.

15. Raccourcis clavier

Touche	Action
`F5`	Rafraîchir
`F6` / `F7` / `F8` / `F9` / `F10` / `F12`	Onglets Commits / Revisions / Branches / Graph / Repos / Log
`F3`	Afficher ou masquer la ligne de commande
`Alt+1` … `Alt+6`	Modes d'affichage des fichiers
`Ctrl+A`	Tout sélectionner
`Ctrl+D` / `Ctrl+Shift+D`	Diff avec l'index / avec le dernier commit
`Ctrl+H`	Historique du fichier
`Ctrl+Enter`	Éditer avec l'éditeur associé
`Ctrl+Shift+G`	Fenêtre Commandes Git
`F7` / `Shift+F7`	Dans un diff : modification suivante / précédente ; dans le résolveur : conflit suivant / précédent
`Alt+1` / `Alt+2`	Dans le résolveur de conflits : garder Ours / Theirs
`Échap`	Dans le champ de filtre : vider le filtre

16. Aide et signalement des bugs

- **Help > Guide de démarrage (français)** : ce guide.
- **Help > Report a Bug / Suggestion** : ouvre la page des issues du projet sur GitHub pour signaler un problème ou proposer une amélioration.
- **Help > Check for Updates** : vérifie s'il existe une nouvelle version ; **Help > About** : version, auteurs et lien vers les issues.
- **Help > Home** et **Getting Started** : la documentation d'origine de GitForce, en anglais.

GitForce a été écrit par Goran Devic et est distribué sous licence GPLv3. Cette version a été modifiée par Germain Philippe : https://github.com/philgerm57/GitForce